

FINAL PROJECT GUIDELINES

APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES

Dynamic Personal Website with Content Management System (CMS)

Project Title

Dynamic Personal Website with User Management and Content Management System

Project Overview

The Dynamic Personal Website with Content Management System (CMS) is a web-based application that allows an individual to professionally present their personal information, skills, achievements, projects, and portfolio through an interactive and modern website.

Unlike traditional static portfolio websites, this project incorporates dynamic functionalities such as user authentication, content management, portfolio management, profile management, and administrative controls using PHP and MySQL.

The project aims to provide students with hands-on experience in developing a complete web application using modern web development practices and technologies commonly used in the industry.

Project Description

Students are required to design and develop a fully functional dynamic personal website that includes:

- Public-facing pages for visitors.
- User registration and authentication.
- Administrative portal for content management.
- Portfolio management system.
- Profile management with image upload functionality.
- Responsive and modern user interface.

The system must use a database to store and retrieve information dynamically rather than relying on hardcoded content.

The project should demonstrate the student's understanding of front-end development, back-end development, database management, security implementation, and software deployment concepts.

Project Objectives

Upon completion of this project, students should be able to:

1. Develop a dynamic web application using PHP and MySQL.
 2. Apply CRUD (Create, Read, Update, Delete) operations using MySQLi procedural or PDO programming.
 3. Implement secure user authentication and session management.
 4. Design and manage relational databases.
 5. Create responsive and user-friendly interfaces.
 6. Apply file upload and image management techniques.
 7. Develop a simple Content Management System (CMS) with CRUD functionality.
 8. Implement role-based access control.
 9. Apply secure coding practices in web development.
 10. Deploy the website in a paid or free hosting environment to make it accessible online.
-

Learning Outcomes

After completing the project, students should be able to demonstrate competencies in:

- Web Application Development
 - Database Integration
 - Validation, Authentication and Authorization
 - Simple Content Management System (CRUD functionality)
 - User Experience Design
 - Responsive Design
 - Emerging Web Technologies
 - Secure Programming Practices
 - Software Documentation
-

Required Technologies

Students must use the following technologies:

Technology	Requirement
Programming Language	PHP
Database	MySQL
Database Access	MySQLi Procedural or PDO
Frontend	HTML5, CSS3, JavaScript
Styling Framework	Bootstrap 5 (Optional but Recommended)
Web Server	Apache (XAMPP/WAMP/LAMP)
Version Control	Optional but Recommended (Git)

Required Functionalities

1. Authentication Module

The system must include:

- User Registration
 - User Login
 - User Logout
 - Session Management
 - Password Encryption
 - Password Validation
-

2. User Profile Management

The system must allow users to:

- Upload profile picture
 - Update profile picture
 - Remove profile picture
 - Edit personal information
 - Change password
-

3. Public Website Pages

The website must include:

- Home Page
 - About Me Page
 - Portfolio Page
 - Contact Page
-

4. Contact Module

Visitors must be able to:

- Send inquiries using a contact form
 - Submit messages to the administrator
-

5. Portfolio Management

Administrators must be able to:

- Add portfolio entries
 - Edit portfolio entries
 - Delete portfolio entries
 - Upload project images
-

6. User Management Module

Administrators must be able to:

- View all users
 - Add users
 - Edit users
 - Delete users
 - Activate or deactivate accounts
-

7. Content Management System (CMS)

Administrators must be able to modify:

- Home Page content

- About Me content
- Portfolio Page content

without editing the source code.

8. Administrative Dashboard

The system must provide:

- Dashboard statistics
 - User counts
 - Portfolio counts
 - Contact message counts
 - Navigation panel
-

Database Requirements

The project must contain at least the following tables:

1. users
2. portfolio
3. cms_pages
4. contact_messages

Additional tables may be added if necessary.

Design Requirements

The website should:

- Have a modern interface.
 - Be mobile responsive.
 - Use proper typography and spacing.
 - Use a professional color palette.
 - Follow usability and accessibility standards.
-

Minimum Security Requirements

Students must implement:

- Password hashing using `password_hash()`
 - Password verification using `password_verify()`
 - Session-based authentication
 - Input validation
 - SQL Injection prevention
 - File upload validation
-

Deliverables

Students must submit:

Source Code

- Complete project folder
- Database SQL file

Documentation

- Project Documentation (Screenshot of modules, database, and file structure)
- The PDF file should be downloadable in the admin account only

Deployment

- The website must be uploaded and deployed online using a free or paid hosting service.
 - Students must provide the live URL of the deployed website.
-

Suggested Database Tables

Table Name	Purpose
users	Stores user information
portfolio	Stores projects and achievements
cms_pages	Stores editable page contents
contact_messages	Stores inquiries from visitors

Grading System

Criteria	Percentage
Functionality and Completeness	40%
Database Design and Integration	15%
User Interface and User Experience	15%
Code Quality and Organization	10%
Security Implementation	10%
Innovation and Additional Features	5%
Documentation	5%
TOTAL	100%

Detailed Rubrics

1. Functionality and Completeness (40%)

Rating	Description
40-36	All required modules are fully functional.
35-30	Minor functionality issues exist.
29-20	Several modules are incomplete.
Below 20	Major functionalities are missing.

2. Database Design and Integration (15%)

Rating	Description
15-13	Database design is efficient and normalized.
12-10	Minor design issues exist.
9-7	Several database problems are present.
Below 7	Poor database implementation.

3. User Interface and User Experience (15%)

Rating	Description
15-13	Modern, responsive, and visually appealing design.
12-10	Acceptable interface with minor issues.
9-7	Interface requires improvement.

Rating	Description
Below 7	Poor user experience.

4. Code Quality and Organization (10%)

Rating	Description
10-9	Clean, modular, and readable code.
8-7	Minor coding issues exist.
6-5	Code organization needs improvement.
Below 5	Poor coding practices observed.

5. Security Implementation (10%)

Rating	Description
10-9	All security requirements implemented properly.
8-7	Minor security issues present.
6-5	Several security measures missing.
Below 5	Serious security vulnerabilities exist.

6. Innovation and Additional Features (5%)

Examples include:

- Dark Mode
 - Email Verification
 - Activity Logs
 - Search Functionality
 - Analytics Dashboard
 - Theme Customization
 - Social Media Integration
-

7. Documentation (5%)

Documentation should include:

- Introduction
- Objectives
- Scope

- System Architecture
 - Database Design
 - Screenshots
 - User Manual
-

Bonus Features (Optional)

Students who implement additional functionalities may receive bonus points at the instructor's discretion.

Examples include:

- Email verification
 - Password reset via email
 - Two-factor authentication
 - Blog module
 - Admin activity logs
 - Notifications
 - Visitor analytics
 - Theme switching
 - API integration
-

Expected Output

For reference, you may visit the link below:

URL: <https://jppalma09.site>

Admin Account: Test Account

Username: admin@example.com

Password: admin123

User Account: Test Account

carlo@example.com

Password: user123